Introducing the Qt Toolkit

What is Qt ?

- Qt is a cross-platform, graphical, application development toolkit that enables you to compile & run applications on Windows, Mac OS X, Linux along with Android and Apple IOS (Qt 5.1.0 onwards).
- Qt is much more than just a Graphical user interface (GUI) toolkit. In fact it has many application libraries built-in that enable network programing, threading, database creation and management, and much more
- Qt is written in C++ and can be used with C++. Since Qt4.8 however, Qt applications may be also written in a new javascript-like language called QML.
- We will primarily focus on developing Qt applications in C++.
- Qt is not written in 'pure C++'. It utilizes fancy macros to implement additional functionality.
- The KDE desktop is written in Qt.
- Another popular GUI toolkit is GTK+ which is written in C.
- This course will focus on the Developing Qt applications on a Windows platform. The process will be very similar on Linux/ Mac platforms.

Things that we will learn in this course

- Qt Library Basics
- Qt Strings and structures
- Basic File I/O in Qt
- Qt Utilities
- Signals, Slots & Events in Qt
- Qt Widgets (Buttons, Labels, e.t.c)
- Qt Layout Managements
- Qt Dialogs
- Qt MainWindow Applications (Single Document interface)
- QSerialPort Library
- QWT Scientific Widgets Library
- QWT Plotting Library
- Drawing in Qt
- Interfacing Qt with Network programming

Things that we will cover in this Lesson

- Introduction to Qt
- Qt Installation
- Getting started with Qt Tool kit
- Building a simple GUI
- Strings & Lists in Qt
- Using qDebug()

Qt Installation

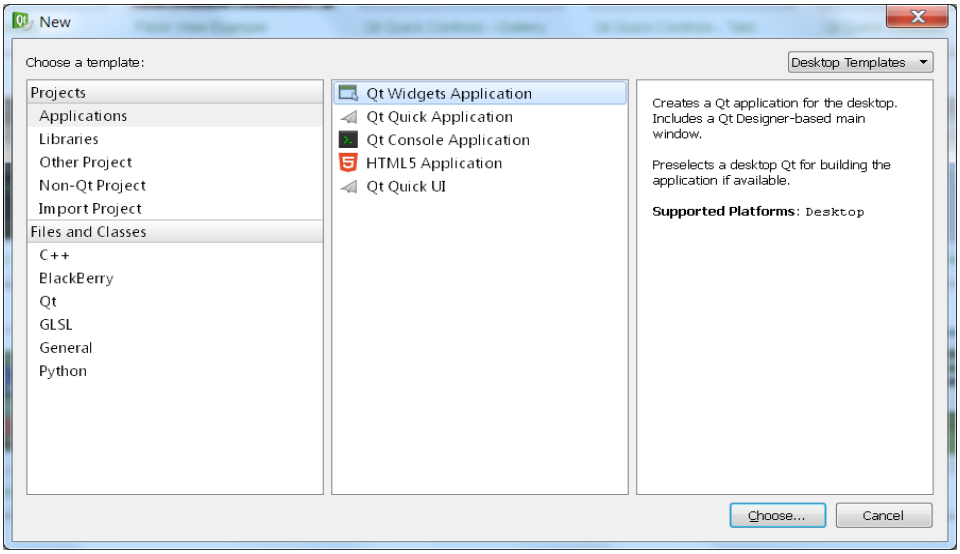
- Download the latest Qt 5.8

package for Windows that includes the Qt5.8 Libraries, the QtCreator IDE and the MingW (GCC Windows port) compiler.

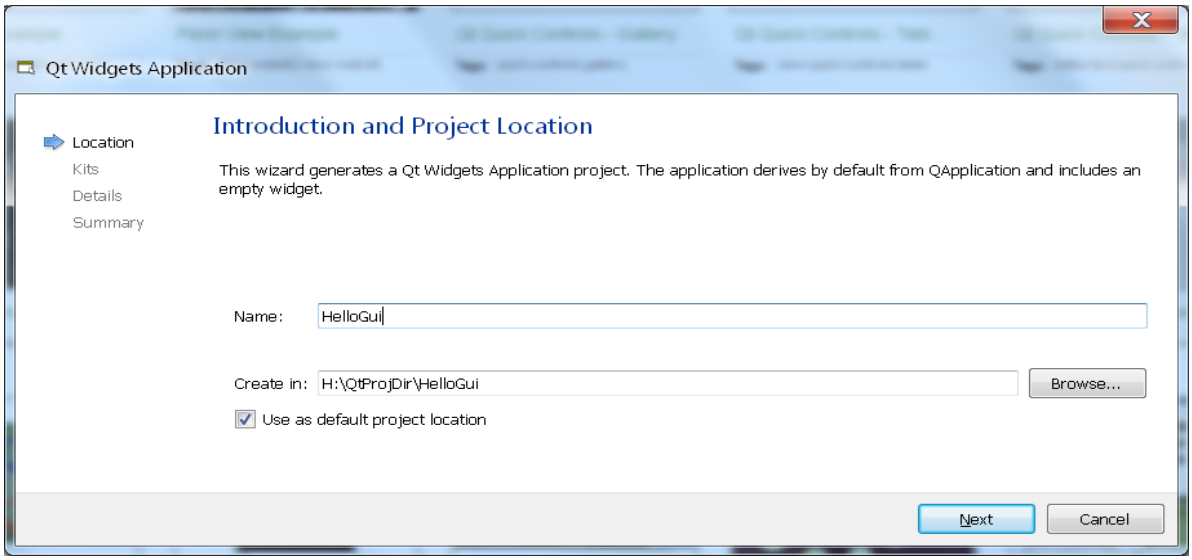
- Run the executable to install.
- Install it in c:\Qt if its the only Qt SDK on your PC or in its own folder i.e. c:\Qt5_8 if you have multiple Qt SDK already installed on your PC.

Creating your first UI application

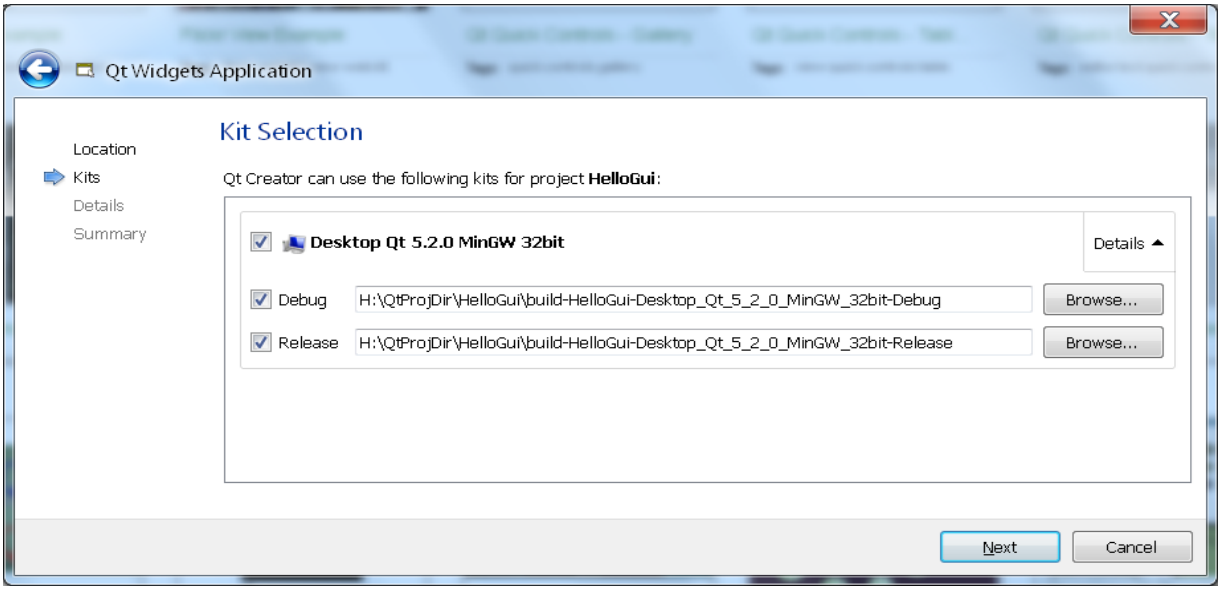
- Create a "QtProjDir" directory on your H Drive. And Create another directory in it called "HelloGui".
- Start the QtCreator IDE
- File-> New File or Project
- Projects →



- In the “Introduction and Project Location Dialog”, name the project “HelloGui” and make sure that the project location is as shown below:

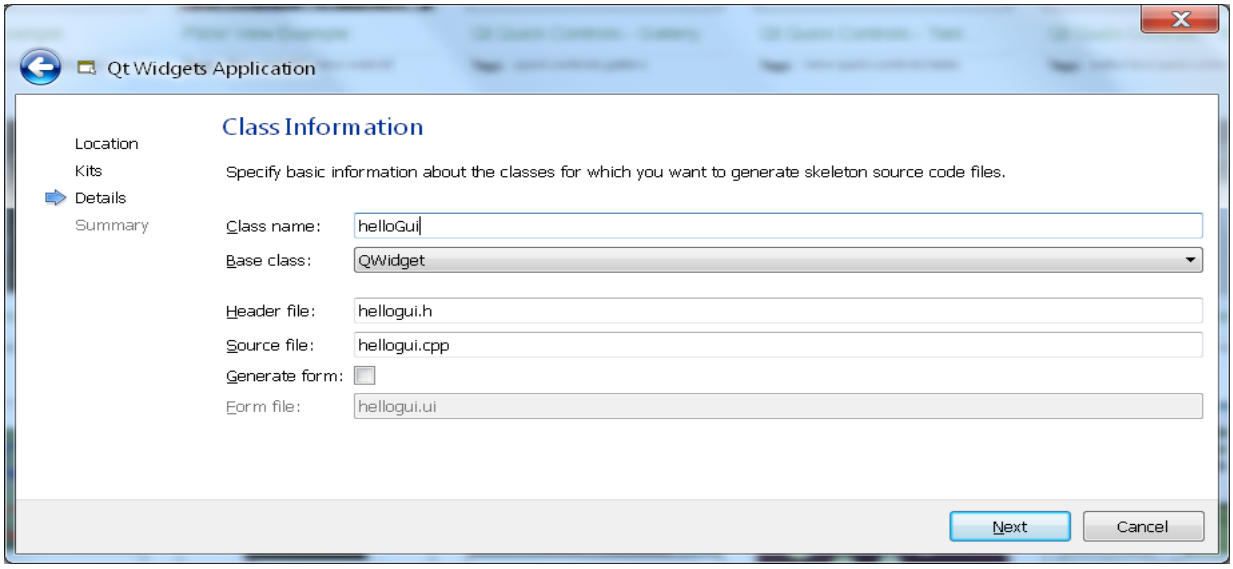


- “In the Kit Selection” dialog accept defaults. It should look like shown below. Note how both Release and Debug folders are inside the project Directory “HelloGui:



- In the “Class Information” Dialog. Change Class name to “helloGui”, Base Class: to “Qwidget”, Header and source files should be helloGui.h & helloGui.cpp. These are the source files that we will use to create our desktop application. The helloGui.h file contains the helloGui class definition. Also make sure that the “Generate form” check box is unchecked. This allows us to create applications in pure C++ as opposed to the graphical desktop app development approach.

- The helloGui.cpp file contains the helloGui class method implementations. The helloGui is an inherited class that inherits all the properties, methods and events of the Qwidget class which is a Qt base clase that implements simple desktop applications or Widgets.



Ex1: What other Qt Base class options are Available? Do a bit of research and briefly describe what types of applications/ components do they implement.

Qwidget: receives mouse, keyboard and other events from the window system
QmainWindow: framework for building an application's user interface. Can add toolbars, docks, menus, etc.
Qdialog: dialog window used for brief communications with the user, can have return values, default buttons. There are various kinds of dialog, for example, Modal dalog: Pops up and blocks input to other windows (like the pop up to ask for a file name from the user).

- In the "project management" dialog accept the defaults and click on finish. Congratulations!!!
- Let's Examine the header file. It contains the class definition for our desktop application "helloGui". **Ex2. What is the purpose of the #ifndef & #endif statements!:**

include guards: checks if the header file has already been defined. If it is not defined #defines it (processor pre-directives).

```
#ifndef HELLOGUI_H
    #define HELLOGUI_H

    #include <Qwidget>      //(1)

    class helloGui : public Qwidget  //(2)
    {



        Q_OBJECT  //(3)

        public:

            helloGui(QWidget *parent = 0); //(4)
```

```
                ~helloGui();                    //(5)
        };

#       endif // HELLOGUI_H
```

1. Include the Qwidget class definitions in the Qwidget header file

2. Define a class “helloGui” derived from (or subclassed from) Qwidget

3. The Q_OBJECT macro must be included (more on this later).

4. Declare the constructor for the class. This constructor must take as a parameter a pointer to the parent object which is set to NULL by default.

5. Declare the destructor for the object.

- Now let's Examine the helloGui.cpp file:

```
#include "hellogui.h"  //(1)
helloGui::helloGui(QWidget *parent)
    : Qwidget(parent)  //(2)
{

}

helloGui::~helloGui() //(3)
{

}
```

1. Include the hellogui.h file

2. Implement the class constructor helloGui. Here is where we will implement our desktop app design

3. Implement the class destructor. Not needed most of the time. If you declare pointers to your own custom classes that point to heap memory with the new keyword. They have to be “freed” here.

- It is important to note that most of our coding will be based in the helloGui.h and helloGui.cpp files. We will rarely modify the main.cpp (Boiler plate code).

```
#include "hellogui.h"

#include <QApplication>

int main(int argc, char *argv[])
{
    QApplication a(argc, argv); //1
```

```
    helloGui w;                 //2
    w.show();//3

    return a.exec();//4
}
```

1. An instance on the Qapplication object is created. This object runs the applications core and the events loop i.e. the GUI application's control flow.
2. The helloGui form object is instantiated
3. the helloGui form is shown
4. On exit the application returns control to the OS.

Modify the helloGui.h file to the following:

```
#ifndef HELLOGUI_H
#define HELLOGUI_H

#include <QWidget>
#include <QLabel> // added
#include <QVBoxLayout> // added

class helloGui : public QWidget
{
    Q_OBJECT

public:
    helloGui(QWidget *parent = 0);
    ~helloGui();

private: //added
    QLabel *label; //added
    QVBoxLayout *vbox; //added

};

#endif // HELLOGUI_H
```

and modify the helloGui.cpp file to the following:

```
#include "hellogui.h"
helloGui::helloGui(QWidget *parent)
    : QWidget(parent)
{

      // label = new QLabel("Hello Qt World",this);
      // label->setFont(QFont("Times New Roman",10));

      QVBoxLayout *vbox = new QVBoxLayout();//added
      label = new QLabel("hello Qt World");
      label->setFont(QFont("Times New Roman", 10));//added
      vbox->addWidget(label);//added
      setLayout(vbox);//added
}
```

```
helloGui::~helloGui()
{

}
```

- These modifications instantiated two additional Qt controls; a Qlabel which simply displays text, and QVboxLayout which arranges controls in a vertical layout i.e. stacks controls vertically. The Qlabel widget is first added to the QVBoxLayout object; "vbox->addWidget(label);" which is then added to the helloGui Form/object; "setLayout(vbox);".
- When you run the code you should get:



- **Ex 3: How can we modify the color of the text and the background of the Qlabel object? Hint: Google it**
- label->setStyleSheet("QLabel { background-color : black; color : green; }");

- **Ex4: How can we add a hyperlink e.g. to the Qlabel object?www.qt-project.org**

```
label->setText("<a href=\"http://www.qtcentre.org/\">hello qt world</a>");
 label->setTextFormat(Qt::RichText);
 label->setTextInteractionFlags(Qt::TextBrowserInteraction);
 label->setOpenExternalLinks(true);
```

Memory Management in Qt

- Notice how the helloGui class constructor took a "pointer to Qwidget" called "parent" as a parameter. This parameter is set to a default value of 0 (NULL)in the class definition in the header file. Furthermore In the main program, the helloGui object 'w' is instantiated without specifying a parameter...meaning that the default (0 or NULL) value is assigned to the 'parent' pointer. This indicates that helloGui is the top level parent in our application.
- Now notice how when we created the Qlabel object label, the second parameter was 'this'. 'this' is a C++ keyword indicating "this instance of the helloGui object" It allows us to define the parent of "label" as the helloGui object. In turn "label" is the child of the helloGui form/object.

- This is important because it means when the parent is closed and its memory is de-allocated, all of its children will also be automatically closed and their memory is de-allocated so long as the parent-child relationship is defined amongst the objects i.e. helloGui is the parent and label is a child.
- When you use a layout object such as the QVBoxLayout object, you do not need to pass a parent when constructing the child widgets as we did for label. The layout will automatically 'reparent' the widgets (using QWidget::setParent()) so that they are children of the widget on which the layout is installed.
- Each Qwidget can have one central layout.
- You can nest layouts using "`addLayout()`" on a layout; the inner layout then becomes a child of the layout it is inserted into.

Finding Qt Documentation

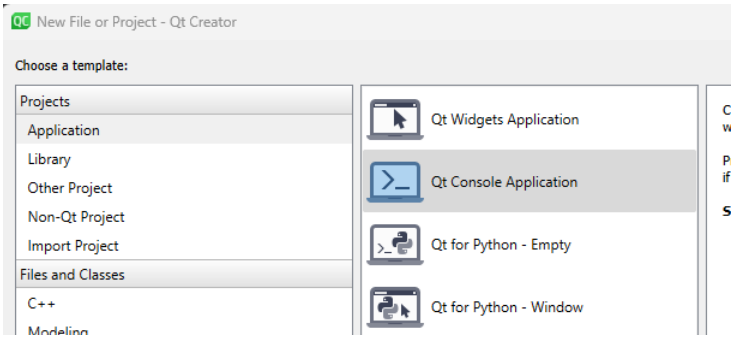
- Qt Classes are very well documented. You can find this documentation on the internet with google. Alternatively you can go to Help-> contents in QtCreator and find all of the documentation on all the classes /objects in Qt.
- **Ex5 Find the documentation for the QLabel class. How many constructors does it implement? How many signals does it have ? And what are they ?**

2 constructors of Qlabel Class:
QLabel(const QString &, QWidget *, Qt::WindowFlags)
QLabel(QWidget *, Qt::WindowFlags)
2 signals of Qlabel class:
linkActivated(const QString &link)
linkHovered(const QString &link)

- Being able to read documentation & examples in Qt is crucial you can find comprehensive documentation and examples at http://doc.qt.io/ & http://doc.qt.io/qt-5/qtexamplesandtutorials.html

Printing Text To Console in Qt

- While you can generally use std::cout to print text to console, the proper 'Qt' way of doing it is with QTextStream objects.



• Create a new console Project in QtCreator ...**Ex6...How ???**
Select console application:

use qmake, everything default (except project name, project folder).

- Copy paste the following code into main.cpp:

```
#include <iostream>
int main(int argc, char *argv[])
{
    std::cout << "Hello Console ..Not Again!!!!\n";

    return 0;
}
```

- This should behave as expected.
- Now copy the following code

```
#include <QTextStream>
#include <QString>
#include <QDebug>
int main()
{
   QString Name;
   QTextStream out(stdout);
   QTextStream in(stdin);
   out << "This is a console application\n";
   out.flush();
   out << "Please enter your name\n";
   out.flush();
   in >> Name;
   out << "Hello "<< Name << "!." << endl;
   out.flush();

   qDebug()<< "use qDebug to send Debugging messages to console in Desktop apps" <<
endl;

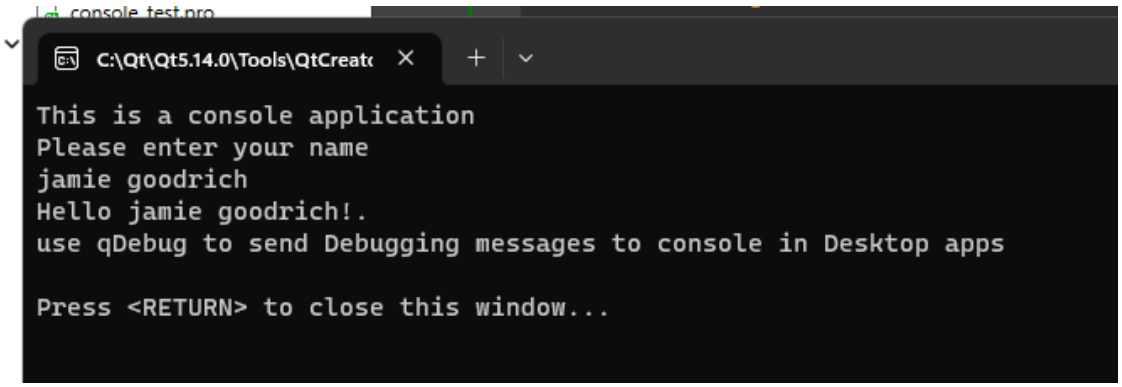
   return 0;

}
```

- Note that we are using Qstring instead of String.
- We then create two QtextStream objects; i)'out' directed to stdout and ii) 'in' directed to stdin. We can then use 'out' and 'in' in the same way we use cout and cin.
- After each time we write to stdout via the out object, we must flush the buffer to see immediate change

- **Ex7 Test the above example by inputting your first name and a second time by using your full name. Does it work the second time? How can you fix this ???**

It does not work the second time (due to the spaces). Name = in.readLine(); By telling the program to read the whole line line it will not stop at the spaces but at the next return.



- Sometimes when we build desktop apps we want to print some debugging messages to the console. We can print such messages using a QTextStream object redirected to stdout. A better and slightly easier way of doing this however is to use the qDebug() function.

QString

- While you can use the standard C++ string class in Qt it is generally preferred to utilize the QString Class

- Run the following example as a console application:

```
#include <QTextStream>
   #include <QString>

   int main(void)
   {
      QTextStream out(stdout);

      QString a = "is";

      a.append(" awesome");
      a.prepend("Music ");

      out << a << endl;
      out << "The a string has " << a.count()
          << " characters" << endl;

      out << a.toUpper() << endl;
      out << a.toLower() << endl;

      return 0;
   }
```

- The “append()” method adds a string after
- The “prepend()” method adds a string before
- The “count()” method returns the length of the string
- the “toUpper()” and “toLower()” convert string to either upper or lower cases.

Ex8 Go through all the Examples found at http://www.zetcode.com/gui/qt4/strings/. In your own words explain the following and provide a simple example:

1. **String Interpolation:** Allows specific string values to be changed dynamically throughout code

```
QString s1 = "There are %1 white roses";
int n = 12;
out << s1.arg(n) << endl;
```

OUTPUT: There are 12 white roses

2. **String Comparison:** compares strings to see if they have the same characters.

```
QString a = "Rain";
QString b = "rain";
QString c = "rain\n";
...
```

OUTPUT: a,b are equal . B and c are not equal.

3. **String modification with remove(), replace() & clear():**

returns a new modified copy of an original string.

```
QString str = "Lovely Season";
```

Remove():

```
str.remove(10, 3);
out << str << endl;
```

OUTPUT:

```
Lovely Sea
Press <RETURN> t
```

replace():

```
str.replace(7, 3, "girl");
out << str << endl;
```

OUTPUT:

```
Lovely Sea
Lovely girl
Press <RETURN>
```

clear():

```
str.clear();
if (str.isEmpty()) {out << "The string is empty" << endl;}
```

OUTPUT:

```
Lovely Sea
Lovely girl
The string is empty
Press <RETURN> to cl
```

Ex9 Is the foreach() statement part of C++ or a Qt add-on? Provide an example of its use

The foreach() keyword is a qt addon to c++ made to simplify making a foreach loop. It simply iterates over elements of an array, vectors or any data set.

```
#include <QTextStream>
int main(void) {

  QTextStream out(stdout);

  int letters = 0;

  QString str = "taco bell is awful";
```

```
 out << str << endl;

 foreach(QChar s, str) {
  if (s.isLetter()) {
    letters++;}
 }

 out << QString("sentence has %1 letters").arg(letters) << endl;
 return 0;
}
```

QList

- Qlists are the Qt equivalent of vectors in C++'s STL.
- They are dynamic arrays that can be grown and modified as needed
- Consider the following example:

```
#include <QList>
#include <QDebug>
#include <QString>

int main()
{

    QList<int> List1;
    QList<QString> List2 = QList<QString>() << "Canada" << "U.S.A" << "Mexico";

    for(int i = 0; i < 6; i++)
        List1.append(i);
    List1.removeOne(4);

    foreach(int n, List1)
        qDebug() << n;

    foreach(QString str, List2)
        qDebug() << str;
    return 0;
}
```

Ex 10. Skim through Qt's documentation on QList. Which member function allows one to find the first occurrence of a particular value?

QList::iterator QList::begin()
Returns an STL-style iterator pointing to the first item in the list.

Finally let's create a slightly more interesting GUI application. This application utilizes a new control; the button. This simple application utilizes Qt's signal and slot mechanism which will be covered in the next set of notes. For now take a look at the code, go through it and ask your teacher to explain aspects of it that are confusing. This app was created by modifying the helloGui app. I highly encourage you to create a new project for this application.

HelloGui.h:

```
#ifndef HELLOGUI_H
#define HELLOGUI_H

#include <QWidget>
#include <QLabel> // added
#include <QVBoxLayout> // added
#include <QPushButton> //added

class helloGui : public QWidget
{
    Q_OBJECT

public:
    helloGui(QWidget *parent = 0);
    ~helloGui();

public slots:
    void changeLabelText(void);

private: //added
    QLabel *label; //added
    QVBoxLayout *vbox; //added
    QPushButton *button;

};

#endif // HELLOGUI_H
```

helloGui.cpp:

```
#include "hellogui.h"
helloGui::helloGui(QWidget *parent)
    : QWidget(parent)
{
      QVBoxLayout *vbox = new QVBoxLayout();//added
      label = new QLabel("hello Qt World");//added
      button = new QPushButton("Click me!");
      label->setFont(QFont("Times New Roman", 10));//added
      vbox->addWidget(label);//added
      vbox->addWidget(button);
      setLayout(vbox);//added

       connect(button, SIGNAL(clicked()), this, SLOT(changeLabelText()));
}

helloGui::~helloGui()
{

}

void helloGui::changeLabelText(void )
{
    label->setText("you changed the label text!!!");
```

```
}
```

main.cpp:

```
#include "hellogui.h"
#include <QApplication>

int main(int argc, char *argv[])
{
    QApplication a(argc, argv);
    helloGui w;
    w.show();

    return a.exec();
}
```

Ex 11. In your own words explain what this code is doing?

Shows a GUI with text and a button that takes in a mouse input as a signal and has a slot that changes the label once the button is pressed.